Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине: «Криптография и сетевая безопасность»

Выполнил:
Студент 4-го курса
Группы АС-66
Головкина В.Д.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: программная реализация симметричного блочного шифра (AES-128 или СТБ 34.101.31 «БелТ») и классических режимов работы (ECB, CBC, CFB, OFB, CTR) с поддержкой алгоритма дополнения блоков (Padding).

Вариант 1

Вариант	Базовый шифр	Режимы работы	Особенности реализации / Фокус тестирования
1	AES-128	ECB + CBC	Реализация PKCS#7 паддинга. Сравнение стойкости к паттернам в данных (ECB vs CBC).

Ход работы

Обязательный теоретический минимум:

- архитектура SP-сетей (на примере AES) и модифицированных сетей Фейстеля (на примере БелТ): генерация раундовых ключей, табличные подстановки (S-box), перестановки и смешивание;
- классические режимы шифрования: Electronic Codebook (ECB), Cipher Block Chaining (CBC), Cipher Feedback (CFB), Output Feedback (OFB), Counter (CTR);
- механизмы дополнения неполных блоков по стандарту PKCS#7;
- побитовые и арифметические операции над байтами и 32-битными словами.

Проект должен состоять из двух логически изолированных модулей:

1. **Блочный шифр:** реализация базового алгоритма (AES-128 или БелТ) согласно варианту. Модуль должен инкапсулировать генерацию раундовых ключей и содержать функции зашифрования/расшифрования строго одного 128-битного блока: encrypt_block(pt_block, key), decrypt_block(ct_block, key).
2. **Режимы шифрования:** модуль, использующий атомарные функции из первого модуля для обработки сообщений произвольной длины. Реализация базового режима ECB (обязательно для всех вариантов) и одного дополнительного режима (CBC, CFB, OFB или CTR). Модуль также должен включать функции наложения и снятия паддинга.

Код:

```
import os
# МОДУЛЬ 1. БАЗОВЫЙ ШИФР AES-128

S_BOX = [
    0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE, 0xD7, 0xAB, 0x76,
    0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4, 0xA2, 0xAF, 0x9C, 0xA4, 0x72, 0xC0,
    0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5, 0xE5, 0xF1, 0x71, 0xD8, 0x31, 0x15,
    0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12, 0x80, 0xE2, 0xEB, 0x27, 0xB2, 0x75,
```

```
    0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B, 0xD6, 0xB3, 0x29,
0xE3, 0x2F, 0x84,
    0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB, 0xBE, 0x39, 0x4A,
0x4C, 0x58, 0xCF,
    0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50, 0x3C,
0x9F, 0xA8,
    0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6, 0xDA, 0x21, 0x10, 0xFF,
0xF3, 0xD2,
    0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64,
0x5D, 0x19, 0x73,
    0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE, 0xB8, 0x14, 0xDE, 0x5E,
0x0B, 0xDB,
    0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3, 0xAC, 0x62, 0x91, 0x95,
0xE4, 0x79,
    0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56, 0xF4, 0xEA, 0x65,
0x7A, 0xAE, 0x08,
    0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD, 0x74, 0x1F, 0x4B,
0xBD, 0x8B, 0x8A,
    0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35, 0x57, 0xB9, 0x86, 0xC1,
0x1D, 0x9E,
    0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E, 0x87, 0xE9, 0xCE, 0x55,
0x28, 0xDF,
    0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0, 0x54,
0xBB, 0x16,
]

INV_S_BOX = [
    0x52, 0x09, 0x6A, 0xD5, 0x30, 0x36, 0xA5, 0x38, 0xBF, 0x40, 0xA3, 0x9E, 0x81, 0xF3,
0xD7, 0xFB,
    0x7C, 0xE3, 0x39, 0x82, 0x9B, 0x2F, 0xFF, 0x87, 0x34, 0x8E, 0x43, 0x44, 0xC4, 0xDE,
0xE9, 0xCB,
    0x54, 0x7B, 0x94, 0x32, 0xA6, 0xC2, 0x23, 0x3D, 0xEE, 0x4C, 0x95, 0x0B, 0x42, 0xFA,
0xC3, 0x4E,
    0x08, 0x2E, 0xA1, 0x66, 0x28, 0xD9, 0x24, 0xB2, 0x76, 0x5B, 0xA2, 0x49, 0x6D, 0x8B,
0xD1, 0x25,
    0x72, 0xF8, 0xF6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xD4, 0xA4, 0x5C, 0xCC, 0x5D, 0x65,
0xB6, 0x92,
    0x6C, 0x70, 0x48, 0x50, 0xFD, 0xED, 0xB9, 0xDA, 0x5E, 0x15, 0x46, 0x57, 0xA7,
0x8D, 0x9D, 0x84,
```

```
    0x90, 0xD8, 0xAB, 0x00, 0x8C, 0xBC, 0xD3, 0x0A, 0xF7, 0xE4, 0x58, 0x05, 0xB8,
0xB3, 0x45, 0x06,
    0xD0, 0x2C, 0x1E, 0x8F, 0xCA, 0x3F, 0x0F, 0x02, 0xC1, 0xAF, 0xBD, 0x03, 0x01,
0x13, 0x8A, 0x6B,
    0x3A, 0x91, 0x11, 0x41, 0x4F, 0x67, 0xDC, 0xEA, 0x97, 0xF2, 0xCF, 0xCE, 0xF0,
0xB4, 0xE6, 0x73,
    0x96, 0xAC, 0x74, 0x22, 0xE7, 0xAD, 0x35, 0x85, 0xE2, 0xF9, 0x37, 0xE8, 0x1C, 0x75,
0xDF, 0x6E,
    0x47, 0xF1, 0x1A, 0x71, 0x1D, 0x29, 0xC5, 0x89, 0x6F, 0xB7, 0x62, 0x0E, 0xAA, 0x18,
0xBE, 0x1B,
    0xFC, 0x56, 0x3E, 0x4B, 0xC6, 0xD2, 0x79, 0x20, 0x9A, 0xDB, 0xC0, 0xFE, 0x78,
0xCD, 0x5A, 0xF4,
    0x1F, 0xDD, 0xA8, 0x33, 0x88, 0x07, 0xC7, 0x31, 0xB1, 0x12, 0x10, 0x59, 0x27, 0x80,
0xEC, 0x5F,
    0x60, 0x51, 0x7F, 0xA9, 0x19, 0xB5, 0x4A, 0x0D, 0x2D, 0xE5, 0x7A, 0x9F, 0x93,
0xC9, 0x9C, 0xEF,
    0xA0, 0xE0, 0x3B, 0x4D, 0xAE, 0x2A, 0xF5, 0xB0, 0xC8, 0xEB, 0xBB, 0x3C, 0x83,
0x53, 0x99, 0x61,
    0x17, 0x2B, 0x04, 0x7E, 0xBA, 0x77, 0xD6, 0x26, 0xE1, 0x69, 0x14, 0x63, 0x55, 0x21,
0x0C, 0x7D,
]

RCON = [0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1B, 0x36]

def xtime(b: int) -> int:
    """Умножение байта b на 2 в GF(2^8)."""
    if b & 0x80:
        return ((b << 1) ^ 0x1B) & 0xFF
    return (b << 1) & 0xFF

def multiply_gf(a: int, b: int) -> int:
    """Умножение двух байтов в GF(2^8)."""
    p = 0
    for _ in range(8):
        if b & 1:
            p ^= a
        hi_bit = a & 0x80
        a = (a << 1) & 0xFF
        if hi_bit:
            a ^= 0x1B
```

```
        b >>= 1
    return p

class AES128:
    """Базовый блочный шифр AES-128 (один 16-байтный блок)."""

    def __init__(self, key: bytes):
        if len(key) != 16:
            raise ValueError("Ключ AES-128 должен быть ровно 16 байт.")
        self.round_keys = self._key_expansion(key)

    # Key Expansion
    def _key_expansion(self, key: bytes):
        w = [list(key[i:i + 4]) for i in range(0, 16, 4)]

        for i in range(4, 44):
            temp = w[i - 1][:]
            if i % 4 == 0:
                temp = temp[1:] + temp[:1]              # RotWord
                temp = [S_BOX[b] for b in temp]         # SubWord
                temp[0] ^= RCON[i // 4]                 # Rcon
            w.append([w[i - 4][j] ^ temp[j] for j in range(4)])

        round_keys = []
        for r in range(11):
            matrix = [[0] * 4 for _ in range(4)]
            for col in range(4):
                word = w[r * 4 + col]
                for row in range(4):
                    matrix[row][col] = word[row]
            round_keys.append(matrix)
        return round_keys

    # Вспомогательные
    @staticmethod
    def _bytes_to_state(block: bytes):
        state = [[0] * 4 for _ in range(4)]
        for i in range(16):
            state[i % 4][i // 4] = block[i]
        return state
```

```
@staticmethod
def _state_to_bytes(state):
    res = bytearray(16)
    for i in range(16):
        res[i] = state[i % 4][i // 4]
    return bytes(res)

def _add_round_key(self, state, rk):
    for r in range(4):
        for c in range(4):
            state[r][c] ^= rk[r][c]

def _sub_bytes(self, state):
    for r in range(4):
        for c in range(4):
            state[r][c] = S_BOX[state[r][c]]

def _inv_sub_bytes(self, state):
    for r in range(4):
        for c in range(4):
            state[r][c] = INV_S_BOX[state[r][c]]

def _shift_rows(self, state):
    state[1] = state[1][1:] + state[1][:1]
    state[2] = state[2][2:] + state[2][:2]
    state[3] = state[3][3:] + state[3][:3]

def _inv_shift_rows(self, state):
    state[1] = state[1][-1:] + state[1][:-1]
    state[2] = state[2][-2:] + state[2][:-2]
    state[3] = state[3][-3:] + state[3][:-3]

def _mix_columns(self, state):
    for c in range(4):
        s0, s1, s2, s3 = state[0][c], state[1][c], state[2][c], state[3][c]
        state[0][c] = xtime(s0) ^ (xtime(s1) ^ s1) ^ s2 ^ s3
        state[1][c] = s0 ^ xtime(s1) ^ (xtime(s2) ^ s2) ^ s3
        state[2][c] = s0 ^ s1 ^ xtime(s2) ^ (xtime(s3) ^ s3)
        state[3][c] = (xtime(s0) ^ s0) ^ s1 ^ s2 ^ xtime(s3)
```

```
    def _inv_mix_columns(self, state):
        for c in range(4):
            s0, s1, s2, s3 = state[0][c], state[1][c], state[2][c], state[3][c]
            state[0][c] = multiply_gf(s0, 0x0E) ^ multiply_gf(s1, 0x0B) ^ multiply_gf(s2, 0x0D)
^ multiply_gf(s3, 0x09)
            state[1][c] = multiply_gf(s0, 0x09) ^ multiply_gf(s1, 0x0E) ^ multiply_gf(s2, 0x0B)
^ multiply_gf(s3, 0x0D)
            state[2][c] = multiply_gf(s0, 0x0D) ^ multiply_gf(s1, 0x09) ^ multiply_gf(s2, 0x0E)
^ multiply_gf(s3, 0x0B)
            state[3][c] = multiply_gf(s0, 0x0B) ^ multiply_gf(s1, 0x0D) ^ multiply_gf(s2, 0x09)
^ multiply_gf(s3, 0x0E)

    # Публичный API
    def encrypt_block(self, pt_block: bytes) -> bytes:
        if len(pt_block) != 16:
            raise ValueError("Размер блока должен быть 16 байт.")
        state = self._bytes_to_state(pt_block)
        self._add_round_key(state, self.round_keys[0])
        for r in range(1, 10):
            self._sub_bytes(state)
            self._shift_rows(state)
            self._mix_columns(state)
            self._add_round_key(state, self.round_keys[r])
        self._sub_bytes(state)
        self._shift_rows(state)
        self._add_round_key(state, self.round_keys[10])
        return self._state_to_bytes(state)

    def decrypt_block(self, ct_block: bytes) -> bytes:
        if len(ct_block) != 16:
            raise ValueError("Размер блока должен быть 16 байт.")
        state = self._bytes_to_state(ct_block)
        self._add_round_key(state, self.round_keys[10])
        for r in range(9, 0, -1):
            self._inv_shift_rows(state)
            self._inv_sub_bytes(state)
            self._add_round_key(state, self.round_keys[r])
            self._inv_mix_columns(state)
        self._inv_shift_rows(state)
```

```
        self._inv_sub_bytes(state)
        self._add_round_key(state, self.round_keys[0])
        return self._state_to_bytes(state)

# МОДУЛЬ 2. РЕЖИМЫ ШИФРОВАНИЯ + PKCS#7

BLOCK_SIZE = 16

def pad_pkcs7(data: bytes, block_size: int = BLOCK_SIZE) -> bytes:
    """PKCS#7: всегда добавляет 1..block_size байт (для кратных — полный блок)."""
    k = block_size - (len(data) % block_size)
    return data + bytes([k]) * k

def unpad_pkcs7(data: bytes, block_size: int = BLOCK_SIZE) -> bytes:
    """
    Снятие PKCS#7 с проверкой. Все ошибки объединены в один тип, чтобы
    не давать атакующему различий (защита от Padding Oracle).
    """
    invalid = ValueError("Invalid PKCS#7 padding")
    if not data or len(data) % block_size != 0:
        raise invalid
    k = data[-1]
    if k < 1 or k > block_size:
        raise invalid
    if data[-k:] != bytes([k]) * k:
        raise invalid
    return data[:-k]

def xor_bytes(a: bytes, b: bytes) -> bytes:
    return bytes(x ^ y for x, y in zip(a, b))

#  ECB

def encrypt_ecb(plaintext: bytes, key: bytes) -> bytes:
    cipher = AES128(key)
    padded = pad_pkcs7(plaintext)
    out = bytearray()
    for i in range(0, len(padded), BLOCK_SIZE):
        out.extend(cipher.encrypt_block(padded[i:i + BLOCK_SIZE]))
    return bytes(out)
```

```
def decrypt_ecb(ciphertext: bytes, key: bytes) -> bytes:
    if len(ciphertext) % BLOCK_SIZE != 0:
        raise ValueError("Длина шифротекста не кратна 16 байтам.")
    cipher = AES128(key)
    padded = bytearray()
    for i in range(0, len(ciphertext), BLOCK_SIZE):
        padded.extend(cipher.decrypt_block(ciphertext[i:i + BLOCK_SIZE]))
    return unpad_pkcs7(bytes(padded))
#CBC
def encrypt_cbc(plaintext: bytes, key: bytes, iv: bytes = None):
    """Возвращает (ciphertext, iv). Если iv не передан — генерируется случайно."""
    if iv is None:
        iv = os.urandom(BLOCK_SIZE)
    if len(iv) != BLOCK_SIZE:
        raise ValueError("IV должен быть 16 байт.")

    cipher = AES128(key)
    padded = pad_pkcs7(plaintext)
    out = bytearray()
    prev = iv
    for i in range(0, len(padded), BLOCK_SIZE):
        block = xor_bytes(padded[i:i + BLOCK_SIZE], prev)
        ct_block = cipher.encrypt_block(block)
        out.extend(ct_block)
        prev = ct_block
    return bytes(out), iv
def decrypt_cbc(ciphertext: bytes, key: bytes, iv: bytes) -> bytes:
    if len(ciphertext) % BLOCK_SIZE != 0:
        raise ValueError("Длина шифротекста не кратна 16 байтам.")
    if len(iv) != BLOCK_SIZE:
        raise ValueError("IV должен быть 16 байт.")
    cipher = AES128(key)
    padded = bytearray()
    prev = iv
    for i in range(0, len(ciphertext), BLOCK_SIZE):
        block = ciphertext[i:i + BLOCK_SIZE]
        decrypted = cipher.decrypt_block(block)
        padded.extend(xor_bytes(decrypted, prev))
        prev = block
```

```
    return unpad_pkcs7(bytes(padded))
# ДЕМОНСТРАЦИЯ
def hexs(b: bytes) -> str:
    return b.hex()

def test_fips197():
    """Эталонный вектор NIST FIPS-197, Приложение C.1 — доказательство корректности AES."""
    print("=" * 70)
    print("1. ТЕСТ-ВЕКТОР FIPS-197 (Приложение C.1)")
    print("=" * 70)
    key = bytes.fromhex("000102030405060708090a0b0c0d0e0f")
    pt = bytes.fromhex("00112233445566778899aabbccddeeff")
    expected = bytes.fromhex("69c4e0d86a7b0430d8cdb78070b4c55a")
    cipher = AES128(key)
    ct = cipher.encrypt_block(pt)
    pt_back = cipher.decrypt_block(ct)
    print(f"Ключ        : {hexs(key)}")
    print(f"Открытый    : {hexs(pt)}")
    print(f"Шифротекст  : {hexs(ct)}")
    print(f"Ожидалось   : {hexs(expected)}")
    print(f"Совпадает   : {ct == expected}")
    print(f"Обратимость : {pt_back == pt}")
    if ct != expected or pt_back != pt:
        raise AssertionError("AES-128 не соответствует FIPS-197!")
    print("☑ FIPS-197 C.1 — пройден\n")
def test_padding():
    print("=" * 70)
    print("2. PKCS#7 PADDING (длины <16, =16, >16)")
    print("=" * 70)
    for n in (0, 5, 15, 16, 17, 32, 43):
        msg = b"A" * n
        padded = pad_pkcs7(msg)
        unpadded = unpad_pkcs7(padded)
        print(f"len={n:3d} -> padded={len(padded):3d}B, "
              f"последний=0x{padded[-1]:02x}, unpad OK={unpadded == msg}")
    print()
def test_roundtrip():
    print("=" * 70)
```

```
    print("3. ШИФРОВАНИЕ/РАСШИФРОВАНИЕ СТРОК (ECB и CBC)")
    print("=" * 70)
    key = bytes.fromhex("2b7e151628aed2a6abf7158809cf4f3c")
    msgs = [
        b"",
        b"Hello",
        b"1234567890123456",
        b"12345678901234567",
        b"The quick brown fox jumps over the lazy dog",
    ]
    for msg in msgs:
        ct_ecb = encrypt_ecb(msg, key)
        pt_ecb = decrypt_ecb(ct_ecb, key)

        ct_cbc, iv = encrypt_cbc(msg, key)
        pt_cbc = decrypt_cbc(ct_cbc, key, iv)

        print(f"len={len(msg):2d}: "
              f"ECB={len(ct_ecb):3d}B OK={pt_ecb == msg} | "
              f"CBC={len(ct_cbc):3d}B OK={pt_cbc == msg}")
    print()

# Демонстрация уязвимости ECB
def make_checkerboard(width=64, height=64, cell=16):
    rows = []
    for y in range(height):
        row = bytearray()
        for x in range(width):
            color = 0xFF if ((x // cell) + (y // cell)) % 2 == 0 else 0x00
            row.append(color)
        rows.append(bytes(row))
    return b"".join(rows)
def save_pgm(path, data, width, height):
    with open(path, "wb") as f:
        f.write(f"P5\n{width} {height}\n255\n".encode())
        f.write(data)
def demo_ecb_pattern():
    print("=" * 70)
    print("4. УЯЗВИМОСТЬ ECB: картинка-шахматка (bitmap)")
    print("=" * 70)
```

```
    key = bytes.fromhex("00112233445566778899aabbccddeeff")
    w = h = 64
    img = make_checkerboard(w, h, cell=16)
    save_pgm("plain.pgm", img, w, h)
    # ECB — отрезаем последний блок паддинга, чтобы размер совпал с картинкой
    ct_ecb = encrypt_ecb(img, key)
    save_pgm("ecb.pgm", ct_ecb[:-BLOCK_SIZE], w, h)
    # CBC
    ct_cbc, iv = encrypt_cbc(img, key)
    save_pgm("cbc.pgm", ct_cbc[:-BLOCK_SIZE], w, h)
    # Подсчёт уникальных блоков — числовое доказательство уязвимости
    def unique_blocks(data: bytes) -> int:
        return len({data[i:i + BLOCK_SIZE] for i in range(0, len(data), BLOCK_SIZE)})
    total_blocks = (w * h) // BLOCK_SIZE
    u_ecb = unique_blocks(ct_ecb[:-BLOCK_SIZE])
    u_cbc = unique_blocks(ct_cbc[:-BLOCK_SIZE])
    print(f"Сохранено: plain.pgm, ecb.pgm, cbc.pgm ({w}x{h}, шахматка {16}px)")
    print(f"Всего блоков данных          : {total_blocks}")
    print(f"Уникальных блоков в ECB      : {u_ecb}   <-- утечка паттерна")
    print(f"Уникальных блоков в CBC      : {u_cbc}")
    print("Вывод: в ECB одинаковые блоки открытого текста дают одинаковые")
    print("шифроблоки -> структура изображения сохраняется. В CBC каждый")
    print("блок уникален за счёт сцепления.\n")
    # Текстовое сравнение на повторяющихся данных
    print("Дополнительно — повторяющиеся 16-байтные блоки текста:")
    data = b"A" * 16 * 4
    ecb_ct = encrypt_ecb(data, key)
    cbc_ct, _ = encrypt_cbc(data, key)
    print("ECB блоки:")
    for i in range(0, len(ecb_ct) - BLOCK_SIZE, BLOCK_SIZE):  # без блока паддинга
        print(f"  {ecb_ct[i:i+16].hex()}")
    print("CBC блоки:")
    for i in range(0, len(cbc_ct) - BLOCK_SIZE, BLOCK_SIZE):
        print(f"  {cbc_ct[i:i+16].hex()}")
    print()
def test_padding_oracle_safety():
    """Проверка, что unpad не различает типы ошибок (одинаковое сообщение)."""
    print("=" * 70)
    print("5. ПРОВЕРКА UNPAD (все некорректные паддинги -> одно исключение)")
    print("=" * 70)
```

```
    bad_inputs = {
        "неверная длина": b"\x00" * 15,
        "k=0": b"\x00" * 15 + b"\x00",
        "k>16": b"\x00" * 15 + b"\x11",
        "несовпадение байтов": b"\x00" * 15 + b"\x05",  # говорит 5, но перед ним нули
    }
    for name, data in bad_inputs.items():
        try:
            unpad_pkcs7(data)
            print(f"  {name}: ОШИБКА — исключение не брошено!")
        except ValueError as e:
            print(f"  {name}: ValueError('{e}')")
    print()
def main():
    test_fips197()
    test_padding()
    test_roundtrip()
    demo_ecb_pattern()
    test_padding_oracle_safety()
    print("=" * 70)
    print("ВСЕ ТЕСТЫ ЗАВЕРШЕНЫ")
    print("=" * 70)

if __name__ == "__main__":
    main()
```

```
======================================================================
1. ТЕСТ-ВЕКТОР FIPS-197 (Приложение C.1)
======================================================================
Ключ         : 000102030405060708090a0b0c0d0e0f
Открытый     : 00112233445566778899aabbccddeeff
Шифротекст   : 69c4e0d86a7b0430d8cdb78070b4c55a
Ожидалось    : 69c4e0d86a7b0430d8cdb78070b4c55a
Совпадает    : True
Обратимость : True
✅ FIPS-197 C.1 — пройден

======================================================================
2. PKCS#7 PADDING (длины <16, =16, >16)
======================================================================
len=  0 -> padded= 16B, последний=0x10, unpad OK=True
len=  5 -> padded= 16B, последний=0x0b, unpad OK=True
len= 15 -> padded= 16B, последний=0x01, unpad OK=True
len= 16 -> padded= 32B, последний=0x10, unpad OK=True
len= 17 -> padded= 32B, последний=0x0f, unpad OK=True
len= 32 -> padded= 48B, последний=0x10, unpad OK=True
len= 43 -> padded= 48B, последний=0x05, unpad OK=True

======================================================================
3. ШИФРОВАНИЕ/РАСШИФРОВАНИЕ СТРОК (ECB и CBC)
======================================================================
len= 0: ECB= 16B OK=True | CBC= 16B OK=True
len= 5: ECB= 16B OK=True | CBC= 16B OK=True
len=16: ECB= 32B OK=True | CBC= 32B OK=True
len=17: ECB= 32B OK=True | CBC= 32B OK=True
len=43: ECB= 48B OK=True | CBC= 48B OK=True
```

```
======================================================================
4. УЯЗВИМОСТЬ ECB: картинка-шахматка (bitmap)
======================================================================
Сохранено: plain.pgm, ecb.pgm, cbc.pgm (64x64, шахматка 16px)
Всего блоков данных           : 256
Уникальных блоков в ECB       : 2   <-- утечка паттерна
Уникальных блоков в CBC       : 256
Вывод: в ECB одинаковые блоки открытого текста дают одинаковые
шифроблоки -> структура изображения сохраняется. В CBC каждый
блок уникален за счёт сцепления.

Дополнительно — повторяющиеся 16-байтные блоки текста:
ECB блоки:
  5301d1251af08a9ea49cf85982d8df6e
  5301d1251af08a9ea49cf85982d8df6e
  5301d1251af08a9ea49cf85982d8df6e
  5301d1251af08a9ea49cf85982d8df6e
CBC блоки:
  44e9e42ad2b1e7a34c9c4c10dae217d3
  984266e577ba8bd27a2acb9152d2d6c6
  aada680294ecc65d2b170086098805a7
  304a5a77d008cd5652bf338788174a3b

======================================================================
5. ПРОВЕРКА UNPAD (все некорректные паддинги -> одно исключение)
======================================================================
  неверная длина: ValueError('Invalid PKCS#7 padding')
  k=0: ValueError('Invalid PKCS#7 padding')
  k>16: ValueError('Invalid PKCS#7 padding')
  несовпадение байтов: ValueError('Invalid PKCS#7 padding')

======================================================================
ВСЕ ТЕСТЫ ЗАВЕРШЕНЫ
======================================================================
```

Вывод: в ходе лабораторной работы я реализовала симметричный блочный шифр (AES-128 или СТБ 34.101.31 «БелТ») и классические режимы работы (ECB, CBC, CFB, OFB, CTR) с поддержкой алгоритма дополнения блоков (Padding).